

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

## Loading the datasets

```
orders = pd.read_csv("olist_orders_dataset.csv")
reviews = pd.read_csv("olist_order_reviews_dataset.csv")
customers = pd.read_csv("olist_customers_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
translation = pd.read_csv("product_category_name_translation.csv")
order_items = pd.read_csv("olist_order_items_dataset.csv")
```

## Inspecting the data

```
orders.head()
# orders.info()
# reviews.head()
# customers.head()
# products.head()
```

[Show hidden output](#)

## Converting the date columns

```
orders['order_estimated_delivery_date'] = pd.to_datetime(orders['order_estimated_delivery_date'])
orders['order_delivered_customer_date'] = pd.to_datetime(orders['order_delivered_customer_date'])
orders.head()
```

[Show hidden output](#)

## Filtering the delivered orders

```
orders = orders[orders['order_status'] == 'delivered']
orders['order_status'].value_counts()
```

[Show hidden output](#)

## Removing duplicates

```
reviews = reviews.drop_duplicates(subset='order_id')
```

## Joining Orders and Reviews

```
orders_reviews = pd.merge(
    orders,
    reviews,
    on='order_id',
    how='left'
)
orders_reviews.head()
```

[Show hidden output](#)

```
## Joining the Customer table with the Orders_reviews

master_data = pd.merge(
    orders_reviews,
    customers,
    on='customer_id',
    how='left'
)
master_data.head()
```

[Show hidden output](#)

```
## Checking for missing Review scores

master_data['review_score'].isnull().sum()

np.int64(646)
```

```
## Calculating the delivery delay

master_data['days_difference'] = (
    master_data['order_delivered_customer_date'] -
    master_data['order_estimated_delivery_date']
).dt.days
# master_data.head()
```

```
## Classifying the delivery status(on Time, Late and Super Late)

def classify_delivery(days):

    if days <= 0:
        return "On Time"

    elif days <= 5:
        return "Late"

    else:
        return "Super Late"
```

```
## Applying the classifying function and then verifying it

master_data['delivery_status'] = master_data['days_difference'].apply(classify_delivery)
# master_data.head()
master_data['delivery_status'].value_counts()
```

[Show hidden output](#)

```
## Counting the total orders per state
```

```
state_total_orders = master_data.groupby('customer_state').size()
state_total_orders.head()
```

[Show hidden output](#)

```
## Counting Late orders per state(ie. late and super late)

late_orders = master_data[
    master_data['delivery_status'] != "On Time"
]

late_by_state = late_orders.groupby('customer_state').size()
late_by_state.head()
```

[Show hidden output](#)

```
## Building comparison table

state_analysis = pd.DataFrame({
    'total_orders': state_total_orders,
    'late_orders': late_by_state
})

state_analysis.head()
```

[Show hidden output](#)

```
## Calculating the late percentage

state_analysis['late_percentage'] = (
    state_analysis['late_orders'] /
    state_analysis['total_orders']
) * 100

state_analysis['late_percentage'] = state_analysis['late_percentage'].round(2)
state_analysis = state_analysis.fillna(0)
```

```
## Sorting the results(worst_performing state)

state_analysis = state_analysis.sort_values(
    by='late_percentage',
    ascending=False
)
```

```
## Inspecting the results

state_analysis.head(10)
```

[Show hidden output](#)

```
## Creating a Visulization(using matplotlib)

state_analysis['late_percentage'].plot(
```

```
        kind='bar',
        figsize=(12,6)
    )

plt.title("Percentage of Late Deliveries by State")
plt.xlabel("State")
plt.ylabel("Late Delivery Percentage")
plt.show()
```

[Show hidden output](#)

```
## Checking if Review score exist in my data
```

```
master_data['review_score'].head()
```

[Show hidden output](#)

```
## Calculating the Average Review Score by the delivery status
```

```
review_analysis = master_data.groupby('delivery_status')['review_score'].mean()
```

```
review_analysis = review_analysis.reset_index()
review_analysis.round(2)
```

[Show hidden output](#)

```
## Creating a visualization(bar chart)
```

```
sns.barplot(
    data=review_analysis,
    x='delivery_status',
    y='review_score'
)
```

```
plt.title("Average Review Score by Delivery Status")
plt.xlabel("Delivery Status")
plt.ylabel("Average Review Score")
plt.show()
```

[Show hidden output](#)

```
## Calculating the correlation between delay days and review score
## (negative correlaton shows more delay => lower review score)
```

```
master_data[['days_difference', 'review_score']].corr()
```

[Show hidden output](#)

```
# order_items = pd.read_csv("olist_order_items_dataset.csv")
```

```
master_data = pd.merge(
    master_data,
    order_items[['order_id', 'product_id']],
```

```
    on='order_id',
    how='left'
)
master_data.head()
```

[Show hidden output](#)

```
## Now we do the Language Translation
## Merging the products table with the master data
```

```
master_data = pd.merge(
    master_data,
    products[['product_id', 'product_category_name']],
    on='product_id',
    how='left'
)
```

```
## Merging the English translator
```

```
master_data = pd.merge(
    master_data,
    translation,
    left_on='product_category_name',
    right_on='product_category_name',
    how='left'
)
```

```
## Renaming the translation column for clarity
```

```
master_data.rename(
    columns={'product_category_name_english': 'category_english'},
    inplace=True
)
master_data.head()
```

[Show hidden output](#)

```
## The Challenge aspect
## Calculating the average delay by product category
```

```
category_analysis = master_data.groupby('category_english')['days_difference'].mean().reset_index()
category_analysis = category_analysis.sort_values(by='days_difference', ascending=False)
```

```
## Visualizing the product display
```

```
plt.figure(figsize=(15,6))
sns.barplot(
    data=category_analysis,
    x='category_english',
    y='days_difference'
)
plt.xticks(rotation=90) # vertical labels
plt.title("Average Delivery Delay by Product Category (Days)")
```

```
plt.xlabel("Product Category")
plt.ylabel("Average Delay (Days)")
plt.tight_layout() # prevent label cutoff
plt.show()
```

[Show hidden output](#)

```
## Downloading the master data
```

```
# master_data.to_csv("final_dashboard_dataset.csv", index=False)
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.